



Faculty of Engineering Ain Shams University

CSE241 Object-Oriented Computer Programming – 2nd Semester 2025/2026

This project is a group project, each group will be **5** students. Each group will deliver a written report in addition to the developed code by the end of the project that shows the detailed analysis, design, testing, etc. of the project and the description of the used techniques, problem description, detailed solution, limitations, sample output, references, and any additional information related to the project. Online demo (**YouTube**) video and discussion will be conducted by the end of the project.

Project Deliverable

The following are required to be delivered by the due date, all deliverables must be uploaded to LMS by the due date, and no other methods can be used to send your files:

- **Project code** must be submitted as a zip/rar archive, it must contain all the files that allow us to run it in addition to the source code.
- **Project document** (in .docx format) that contains at least the sections stated above.
- A **GitHub** link (you will provide) has all files and documents mentioned above. "Prepare it well for you :)"
- A **YouTube video** (Demo) that illustrates the project's use.

Project Title: Desktop Hotel Reservation System

Project Overview:

You will design and implement a Desktop Hotel Reservation System in Java, applying the core concepts taught throughout the course. The project is divided into two milestones:

- **Milestone 1** focuses on building a solid, well-structured backend using OOP principles.
- **Milestone 2** focuses on building a fully interactive JavaFX GUI, integrating multi-threading for real-time updates, and implementing basic networking via Java Sockets.

Technology Requirement: The GUI must be implemented using JavaFX (not Swing). All UI screens, layouts, and event handling must use the JavaFX framework.

Team Responsibilities

Each group of 5 students must:

- Divide work fairly and document each member's contribution in the final report.
- Use Git/GitHub collaboratively — commit history will be reviewed.
- All members must be able to explain any part of the project during the discussion session.

Milestone 1: OOP Foundations

[10 marks]

Due Date: 24 April 2026 | Marks: 10

Objective:

Implement the complete backend structure of the Hotel Reservation System using core OOP principles. No GUI is required at this stage, focus entirely on well-designed, functional Java classes.

Key Features:

1. Guest:

1. Model a Guest class using proper OOP principles.

2. Required attributes:

- username (String), password (String, validated), dateOfBirth (LocalDate or Date), balance (double), address (String), gender (**must be an enum**: MALE, FEMALE), roomPreferences (e.g., preferred room type, floor, etc.)

3. Required behaviors:

- Register / Login
- View available rooms, Make, view, and cancel reservations
- Checkout and pay invoice

2. Admin/Receptionist:

- Model Admin and Receptionist classes using OOP principles (consider using inheritance from a common Staff parent class).
- **Required attributes:**
 - username, password, dateOfBirth, role (enum: ADMIN, RECEPTIONIST), workingHours (int or schedule object)
- **Required behaviors:**
 - View all guests, rooms, and reservations
 - Admin: full CRUD on rooms, amenities, and room types
 - Receptionist: manage check-in/check-out

3. Room Types and Amenities:

- Design a RoomType class (e.g., Single, Double, Suite).
- Design an Amenity class (e.g., WiFi, TV, Mini-bar).
- A room should be associated with one RoomType and a list of Amenity objects.
- Admin must be able to Create, Read, Update, and Delete room types and amenities.

4. Reservations and Invoices:

- Design a Reservation class with:
 - Guest reference, room reference
 - Check-in and check-out dates
 - Reservation status (enum: PENDING, CONFIRMED, CANCELLED, COMPLETED)
- Design an Invoice class with:
 - Total amount, payment method (enum: CASH, CREDIT_CARD, ONLINE), payment date.
 - Support multiple payment methods at checkout.

5. Validation:

- Use encapsulation (private fields + getters/setters) throughout all classes.
- Validate all inputs (e.g., no negative balance, valid date ranges, non-empty usernames).
- Throw meaningful custom exceptions where appropriate (e.g., RoomNotAvailableException, InvalidPaymentException).

6. In-Memory Database

- Create a Database class (or HotelDatabase) that acts as the application's data store.
- Use static ArrayList<T> fields for each entity:
 - ArrayList<Guest> guests
 - ArrayList<Room> rooms
 - ArrayList<Reservation> reservations
 - ArrayList<Invoice> invoices
 - etc.
- Pre-populate with dummy data so the system can be tested immediately on launch.

Milestone 1 Deliverables:

- Full Java source code (all backend classes)
- UML Class Diagram showing all classes, relationships, and hierarchies
- Use of **interfaces and/or abstract classes** to define contracts (e.g., Payable, Manageable)
- A short section in the report describing your design decisions

Milestone 2: GUI, Multi-threading, and Network Basics

[10 marks]

Due Date: 30 April 2026 | Marks: 10

Objective:

Enhance the system by building a complete JavaFX-based GUI, adding multi-threading for real-time features, and implementing basic client-server networking using Java Sockets.

Key Features:

- **JavaFX Graphical User Interface (GUI)**
 - Mandatory: Use JavaFX only. Swing is not accepted for this milestone.
 - Design and implement the following screens:
 - Login / Register: Guest and staff login; new guest registration with validation
 - Guest Dashboard: View profile, balance, and active reservations
 - Room Browsing: Browse available rooms with filtering by type, price, amenities
 - Reservation Management: Make a new reservation, view history, cancel a reservation
 - Checkout / Payment: Select payment method, view invoice, confirm payment
- **UI Requirements:**
 - All screens must use proper JavaFX layouts (e.g., BorderPane, GridPane, VBox, HBox).
 - Use FXML files for at least 3 screens (recommended for clean MVC separation).
 - Apply CSS styling to give the application a consistent and professional look.
 - All user actions must display appropriate feedback messages (alerts, dialogs, or status labels).
 - Navigation between screens must be smooth and intuitive (use Scene switching or StackPane layers).

Milestone 2 Deliverables:

- Fully functional JavaFX application (all screens listed above)
- At least 3 FXML layout files
- CSS stylesheet applied to the application
- Test cases (valid and invalid scenarios) documented in the report

Bonus (Maximum 2 marks)

- Multi-threading
 - Use Java threads or JavaFX's Task / Service classes for background operations
 - Required use cases:
 - Real-time room availability updates (refresh room list without freezing the UI).
 - Background loading of data (e.g., loading reservations asynchronously).
 - Never update JavaFX UI components from a non-JavaFX thread — use Platform.runLater() where needed.
- Basic Networking (Client-Server)
 - Implement a simple client-server architecture using Java Sockets.
 - Required use case:
 - A live chat feature allowing a guest to send messages to the receptionist and receive replies in real-time.
 - Implementation requirements:
 - Server must handle multiple clients concurrently using threads.
 - Use ServerSocket and Socket classes.
 - Messages must appear in the GUI chat window in real-time.
- Connecting to a real database instead of in-memory database

Submission Checklist

All items must be uploaded to LMS by the due date. No other submission method is accepted.

- **Source Code:** .zip or .rar archive
- **Project Report:** .docx file
- **GitHub Repository:** Public or shared link - must contain code, report, and UML
- **Demo Video:** YouTube link (unlisted or public) - must demonstrate all features

GitHub Tip: Keep your repository clean and well-organized. Use a proper README.md, meaningful commit messages, and separate branches per milestone. Reviewers will look at your commit history!

Discussion

Marks: 5 (Individual Mark)

A live discussion session will be held after final submission. Every team member must be able to:

- Explain their individual contribution to the project.
- Walk through any part of the code on demand.
- Answer questions about OOP design choices, JavaFX implementation, threading, and networking.

Members who cannot explain their own code will be deducted marks for the discussion portion, regardless of the group's submission quality.

Good luck, teams! Build something you're proud of — this project is a great opportunity to bring everything you've learned in CSE241 to life. Make it clean, make it functional, and most importantly, make it yours!